

Seminarski rad

Prikupljanje i predobrada podataka za mašinsko učenje

Otklanjanje šuma u skupovima podataka za mašinsko učenje

Student:

Danilo Milošević 1732

Profesor:

Prof. Dr. Alekandar Stanimirović

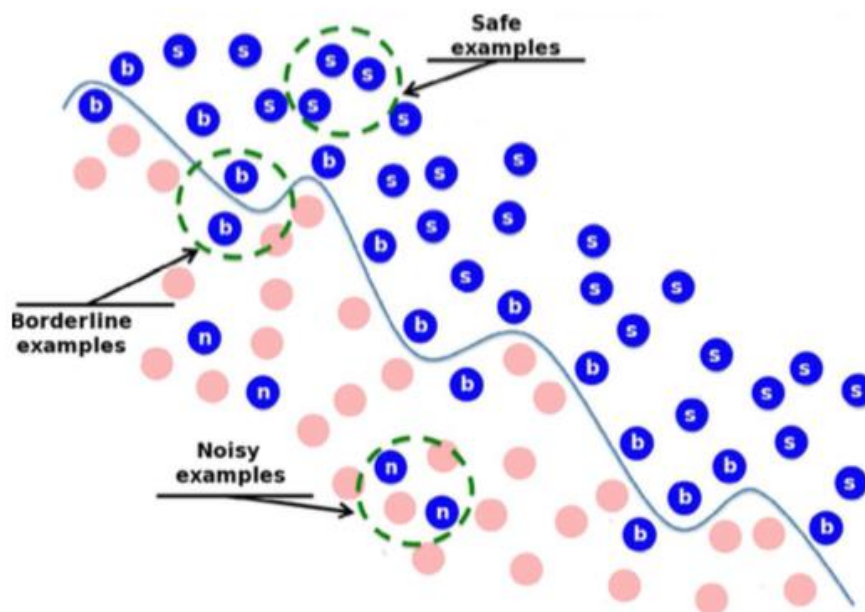
Sadržaj

1. Uvod.....	2
2. Definicija šuma	3
2.1 Šta je šum, vrste šuma	4
2.2 Nasumični šum i uticaj na model	5
2.2 Šum usled broja feature-a i uticaj na model	6
2.2 Outlier šum i uticaj na model	7
2.2 Metode poboljšanja performansi modela kod skupa podataka sa šumom	8
3. Metode otklanjanja šuma	10
3.1 Regularizacija modela	10
3.2 Otklanjanje šuma usled broja feature-a	13
3.3 Otklanjanje outlier	14
3.4 Redukcija nasumičnog šuma	17
3.5 Otklanjanje šuma kod alfanumeričkih podataka	20
3.5.1 Tipovi šuma kod alfanumeričkih podataka	21
3.5.2 Otklanjanje šuma atributa	22
3.5.4 Otklanjanje šuma labela.....	24
3.5.5 Modeli otporni na šum.....	26
3.5.5 Postizanje otpornosti na šum MCS modela kod višeklasne klasifikacije	29
3.5.6 Uvođenje šuma u podacima	30
3.6 Otklanjanje šuma korišćenjem autoencoder-a	30
5. Literatura	32

1. Uvod

Svedoci smo sve bržeg razvoja veštačke inteligencije i mašinskog učenja tokom prošle decenije i godina, koji je predvođen pre svega razvojem sve kompleksnijih modela kao što su konvolucione neuronske mreže, transformeri kao i mnogi drugi. Međutim, kreiranje sve većih i kompleksnijih modela nije dovoljan za postizanje boljih rezultata, već je potrebno i trenirati modele nad kvalitetnim podacima. Kvalitet podataka zavisi od relevantnosti samih podataka za cilj, kompletnosti podataka ali i količine šuma među podacima - šuma koji može biti posledica slučajnih grešaka, grešaka u merenju, nedoslednosti kao i mnogih drugih izvora.

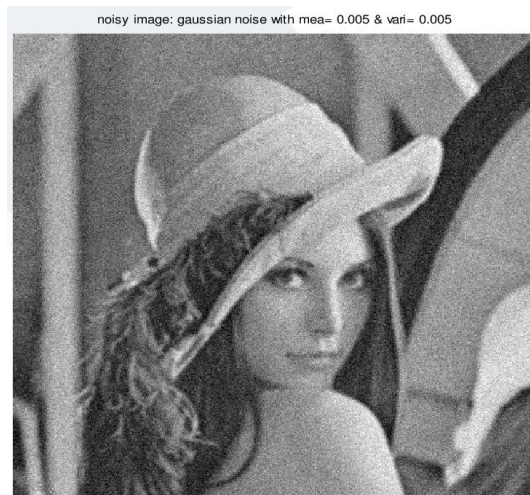
U ovom redu ćemo definisati šta je tačno šum, vrste šuma, zašto je bitno otkloniti šum među podacima kao i algoritme kojima to možemo postići. Za sve metode otklanjanja šuma ćemo dati objašnjenje principa rada algoritma, prednosti i nedostatke kao i kako različiti pristupi otklanjanju šuma mogu poboljšati tačnost i efikasnost modela, smanjujući rizik od pogrešnih predikcija i povećavajući robusnost modela u različitim scenarijima. Uz ovaj rad će biti priložena i praktična primena obrađenih algoritma.



2. Definicija šuma

2.1 Šta je šum, vrste šuma

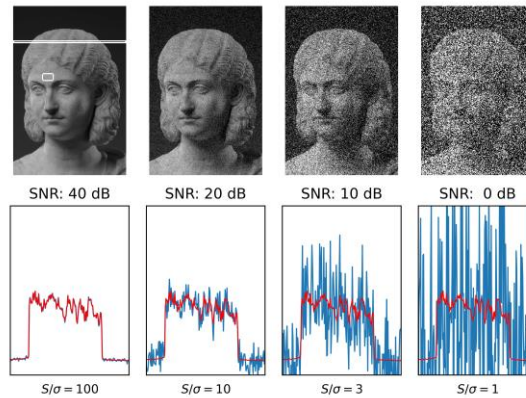
Šum predstavlja nasumične ili nebitne podatke koji se javljaju u nekom skupu podataka, odnosno u signalu. U kontekstu mašinskog učenja šum se ispoljava kao nepredvidive, naizgled nasumične, fluktuacije u skupu podataka koje otežavaju modelu da identifikuje obrasce i odnose među podacima. Iz ovih razloga je ključno adresirati šum u podacima pre treniranja modela, inače izlaz modela može biti loš - garbage in, garbage out.



Postoje više vrsta šuma:

- **Nasumični šum** - najpoznatiji je Gausov šum, koji predstavlja nasumičan šum sa normalnom raspodelom verovatnoće. Pored Gausovog postoje još mnogo vrsta šuma, kao što su beli, crni, roze, Brown-ov šum, Košijev šum i mnogi drugi. Ova vrsta šuma najčešće nastaje usled malih nepreciznosti tokom merenja.
- **Šum usled broja feature-a** - skupovi podataka mogu posedovati šum usled suvišnih atributa u skupu. Zbog ovoga je bitno primeniti redukciju dimenzionalnosti, koju ćemo kasnije spomenuti.
- **Outlier šum** - ovaj šum predstavljaju podaci koji značajno odstupaju od ostalih podataka. Njihova devijacija znatno utiče na sposobnost algoritama da prepoznaju obrasce u podacima.
- **Šum labela** - nastaje kada skup ima netačne ili nekonzistentne labela među podacima i najčešće je posledica ljudske greške tokom dodeljivanja labela.
- **Konceptualni šum** - šum koji nastaje pogrešnim skupljanjem podataka kojim se uvode nekonzistentnosti u skup podataka, čime se povećava bias.
- **Pozadinski šum** - javlja se kod nekih tipova podataka, kao što su snimci, audio zapisi, slike. Primer ovog šuma je šum vetra ili električnih uređaja u audio zapisima koji otežava trening modela za prepoznavanje glasova.

U ovom radu ćemo se fokusirati na metode za otklanjanje prve 4 vrste šuma. U narednim poglavljima pogledaćemo svaku od spomenutih 4 vrste šuma ponaosob i njihov uticaj na modele kao i pregled metoda za poboljšanje performansi modela kod skupa podataka sa šumom.



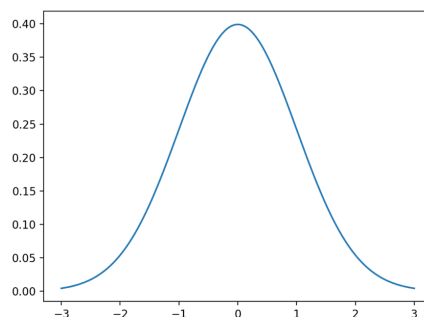
2.2 Nasumični šum i uticaj na model

U ovom poglavlju ćemo razmatrati aditivni šum - nasumičan šum koji se dodaje originalnom signalu. Izvor ovog šuma su najčešće nepreciznosti tokom merenja i u maloj količini je često i korisna pojava koja može poslužiti za regularizaciju neuronskih mreža kako bi se smanjio overfit. Pored toga šum može poslužiti i za augmentaciju podataka. Međutim u slučajevima kada je šum previše zastupljen, može pogoršati rezultate pa je potrebno ukloniti šum.

Nasumičan šum predstavlja nasumično generisane podatke sa određenom funkcijom raspodelom verovatnoća. Tip nasumičnog šuma je određen upravo ovom funkcijom. Najpoznatiji nasumičan šum, s obzirom na često pojavljivanje u prirodi i različitim procesima, jeste Gausov šum koji prati normalnu, Gausovu, raspodelu.

$$\varphi(z) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

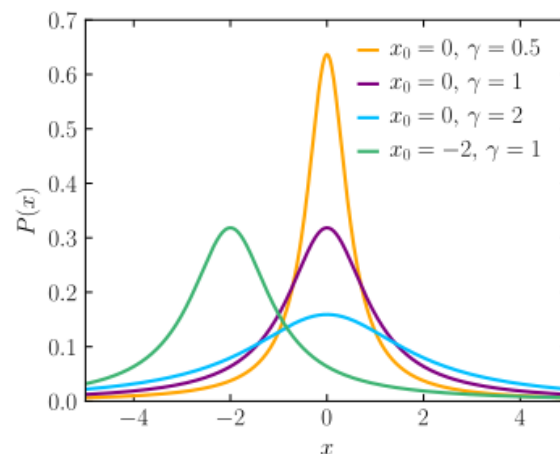
pri čemu su σ i μ , standardna devijacija i srednja vrednost podataka, respektivno, koji su kod normalne raspodele jednaki 0 i 1



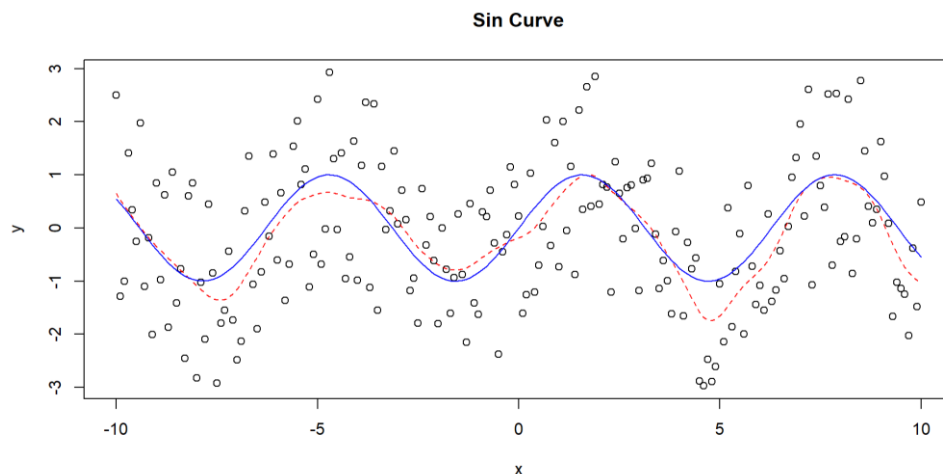
Pored toga, često prisutna vrsta nasumičnog šuma je Košijev šum koji je čest kod medicinskih i biomedicinskih podataka. Košijev šum ima raspodelu verovatnoća

$$\varphi(z) = \frac{1}{\pi\gamma \left[1 + \left(\frac{x - x_0}{\gamma} \right)^2 \right]}$$

pri čemu je x_0 mediana i γ prosečna apsolutna devijacija.



Ispod možemo da vidimo uticaj šuma na treniranje modela. Skup podataka je generisan sinusoidnom funkcijom a zatim je dodat šum skupu. U ovom slučaju model nije u stanju da zaključi da podaci prate sinusoidalnu raspodelu usled šuma, tj. model overfit-uje podatke sa šumom. Smanjivanjem šuma ili regularizacijom može se smanjiti overfit u ovom slučaju.

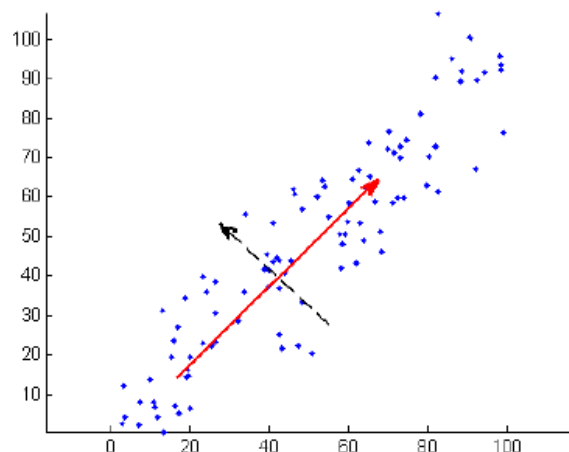


2.2 Šum usled broja feature-a i uticaj na model

Pored nasumičnog šuma, razmotrićemo i šum koji nastaje usled broja atributa, odnosno feature-a u skupu podataka. Ovaj šum ne nastaje greškama u merenju, već kada skup ima podatke koji nisu relevantni za cilj koji model treba da postigne, što je česta pojava kod realnih skupova podataka.

Kao i kod nasumičnog šuma, prisutnost ove vrste šuma može da prouzrokuje da model overfit-uje, odnosno da model zapamti i šum u skupu podataka umesto da prepozna obrasce koji se javljaju među podacima. Zato je često potrebno prvo izvršiti redukciju dimenzionalnosti kojom se smanjuje broj atributa, odnosno dimenzije ulaznih podataka. Redukcija dimenzionalnosti se takođe vrši kako bi se izbeglo "prokletstvo dimenzionalnosti" koje nastaje usled smanjene gustine podataka u visokim dimenzijama i time otežanog treninga modela, međutim ovde će biti korišćena samo u kontekstu smanjenja šuma.

Jedan primer ovakvog šuma možemo videti na slici ispod.



Najveća varijansa (raznovernost) u podacima je u smeru crvenog vektora, dok je varijansa mnogo manja duž smera crnog vektora. U visoko dimenzionalnim skupovima podataka varijansa duž pojedinih smerova može biti još manja nego na slici i u tom slučaju dodatni podaci odnosno dimenzije se mogu smatrati šumom, s obzirom da ne donose nove, bitne podatke i izazvaće da model overfituje ukoliko ne koristimo regularizaciju.

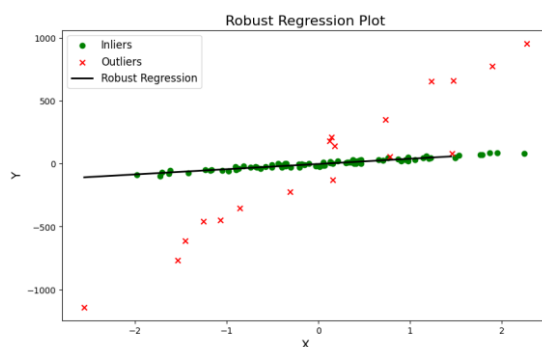
Kako bi izbegli probleme koje izazivaju dodatni atributi, možemo primeniti algoritme za redukciju dimenzionalnosti koje ćemo kasnije obraditi, ili primeniti regularizaciju kako bi minimizovali uticaj šuma koji potiče od dodatnih dimenzija.

2.2 Outlier šum i uticaj na model

Poslednja vrsta šuma koju ćemo razmotriti u ovom radu je outlier šum. Outlier šum nastaje kada neki od podataka u skupu se dosta razlikuje od ostataka skupa i može da nastane kao posledica grešaka u podacima, šuma ali i specijalne slučajeve u podacima.

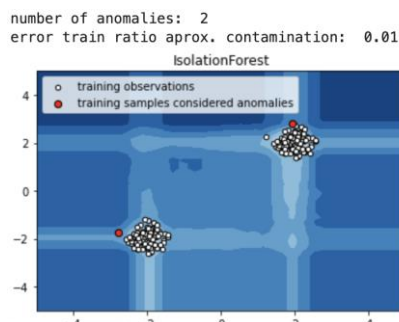
Ova vrsta šuma je pogotovo teška za detektovanje i otklanjanje, s obzirom da je teško razlikovati šta je outlier a šta validan podatak - ako imamo puno outlier-a, da li su oni outlieri ili podaci nisu prikupljeni kako treba, na primer neka klasa koja nije dovoljno zastupljena u podacima? Pored toga, koliko se treba outlier razlikovati od ostalih podataka da bi se smatrao outlierom? Ovo će biti razmotreno kasnije kada radimo sa algoritmima za otklanjanje outliera.

Prisutnost outliera može značajno da pogorša rezultate modela.



Na slici iznad imamo skup podataka koji sadrži outliere. U ovom slučaju je primenjen algoritam za otklanjanje outliera kao i regularizacija pa je model otporan na outlier-e, ali bez ovog preprocesiranja model bi imao znatno gore rezultate, s obzirom da bi outlieri primorali da regresiona prava prava promeni nagib. Slika je demonstracija koliko ova vrsta šuma može pogoršati performanse, kao i činjenicu da je potrebno razmotriti koliko treba biti "osetljiv" kod odlučivanja koje instance su inlier-i a koje outlier-i. Podaci koji se nalaze daleko od linije mogu se odbaciti, ali da li outlier-e čine i podaci koji su vrlo blizu regresionoj liniji.

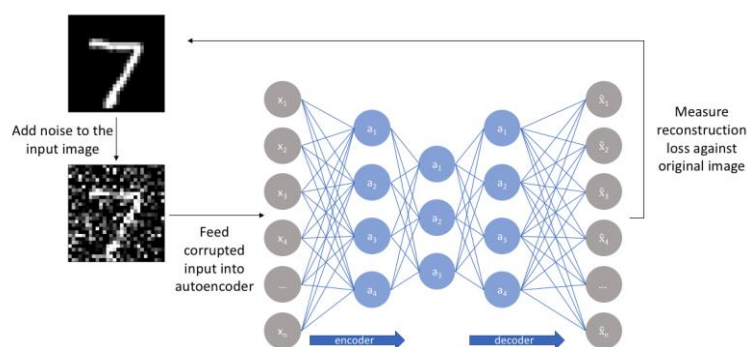
Postoje mnogi algoritmi za uklanjanje outlier-a (ispod možemo videti otklanjanje outliera pomoću Isolation Foresta), koje ćemo u narednom poglavlju razmotriti.



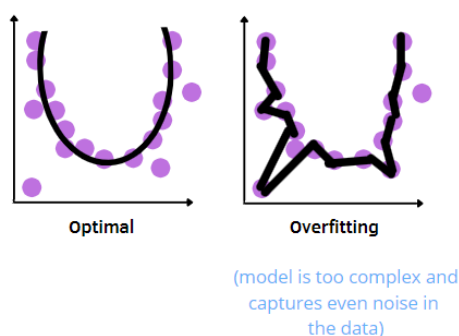
2.2 Metode poboljšanja performansi modela kod skupa podataka sa šumom

Za poboljšanje performansi modela kod noisy skupova podataka koristićemo tri tehnike - otklanjanje šuma, regularizacija i tehnike za kompenzaciju.

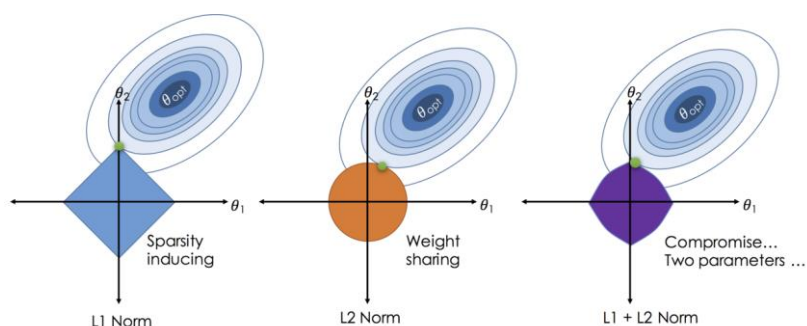
Prva metoda se odnosi na samo uklanjanje šuma, što će činiti većinu daljeg rada. Otklanjanje šuma može da se odnosi na nasumičan šum, šum usled outlier-a kao i šum usled broja dimenzija, zbog čega ćemo svaku oblast obraditi ponaosob - algoritme koji se koriste, prednosti i nedostatke. Kao posebno korisna metoda za sva od ova 3 zadatke se izdvajaju autoenkodori koje ćemo posebno obraditi. Sama struktura autoenkodera dovodi do redukcije dimenzionalnosti, a s obzirom da se oni mogu trenirati da otklanjaju šum i rezultati koristiti za određivanje outliera [], mogu poslužiti kao dobar alat za sva tri zadatka. Slika ispod demonstrira rad autoenkodera za otklanjanje šuma.



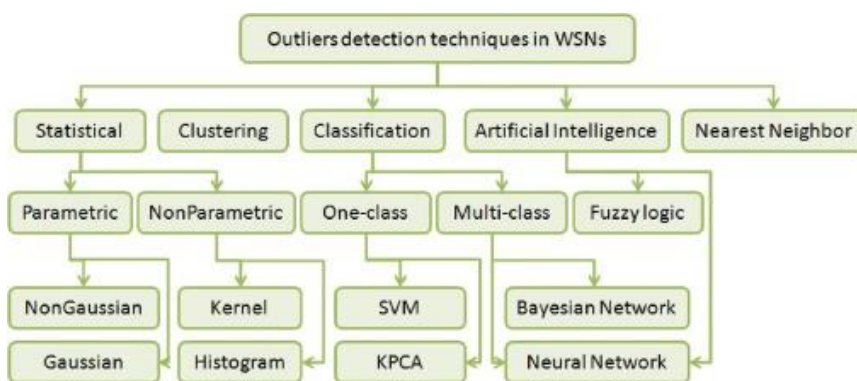
Često je nemoguće potpuno otkloniti šum, s obzirom da šum često ne prati savršeno Gausovu ili Košijevu raspodelu, tako da nije uvek lako modelovati i prepoznati šum. Zato je potrebno dodatno učiniti modele otpornim na šum u podacima što je moguće ostvariti regularizacijom. Ovom metodom se obezbeđuje da model ne "uči napamet" tako što se meri greške modela dodaje još jedan termin koji se obično računa na osnovu koeficijenata (težina) modela čime se model čini otpornijim na šum. Pored toga, poznata metoda za regularizaciju kod neuronskih mreža je dropout, koju ćemo posebno obraditi. Ispod možemo videti efekat šuma u modelu bez regularizacije (desna slika) kao i sa regularizacijom (slika levo).



Najpoznatije metode regularizacije su l_1 i l_2 , koje vidimo na slici ispod.



Poslednja metoda se odnosi na otklanjanje outlier-a. Čak i sa regularizacijom i otklanjanjem šuma, neki podaci previše odstupaju od drugih što može izazvati da model pogrešno nauči obrasce među podacima. Postoji mnogo metoda za otklanjanje outlier-a i one mogu biti bazirane na statistici, clustering-u, algoritmima za klasterizaciju kao i veštačkim neuronskim mrežama.

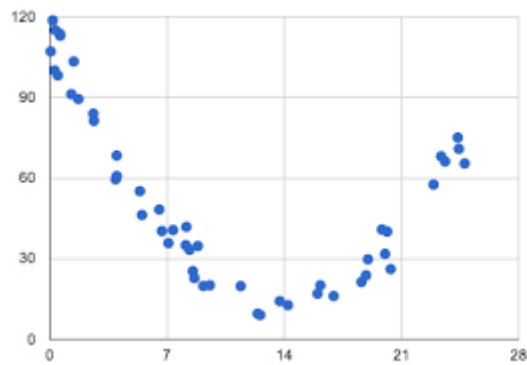


3. Metode otklanjanja šuma

3.1 Regularizacija modela

Od pomenutih metoda, prvo ćemo obraditi regularizaciju modela. Regularizacija modela je nastala usled problema "overfitting"-a, gde se model previše fokusira na podatke u trening skupu, te umesto da nauči raspodelu odnosno šablon među podacima on takođe memoriše i sitnije nepravilnosti, odnosno i šum. To dovodi da model slabije generalizuje, odnosno gore se ponaša sa novim podacima koje ranije nije video.

U realnosti je nemoguće savršeno skupiti podatke, svako merenje i svaki senzor uvodi neki šum, tj neke nepravilnost. Na primer, iako postoji određena zavisnost (linearna, kvadratna) među podacima, usled ljudske greške ili greške instrumenata, uvek će doći do odstupanja, pa će model teže primetiti šablone među podacima. Na slici ispod možemo videti primer toga.

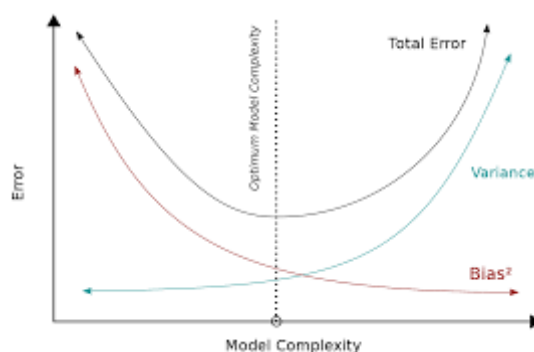


Podaci očigledno prate krivu $y = x^2$, međutim i šum je prisutan u određenoj meri. Ovde se javlja problem bias-variance trade-offa. Kako bi trenirali model, prvo moramo da izaberemo koliko slobode model ima, tj koliko može imati parametara za učenje a time i koliko kompleksne podatke može modelovati. Tu se javlja problem bias-a, koji se može prevesti kao predrasuda, koji nastaje usled našeg izbora stepena slobode koji model ima.

Kod treninga možemo pretpostaviti da podaci prate linearnu zavisnost i izabrati linearnu regresiju - time uvodimo visok bias, s obzirom da ne dajemo slobodu modelu da uoči bilo koju kompleksniju zavisnost (polinomnu, kvadratnu...). Međutim, time postizemo manju varijansu, model je manje osetljiv na šum, odnosno male nepravilnosti neće biti modelovane.

Sa druge strane, ako odlučimo da model ima velik stepen sloboda, odnosno puno parametara (neuronske mreže) onda će model moći da prepozna kako jednostavne tako i kompleksne šablone i zavisnosti među podacima. U tom slučaju je bias nizak, jer ne uvodimo predrasudu kakvu zavisnost prate podaci. Tu je loša strana što je varijansa visoka - model precizno modelu-je i šum među podacima.

Ovo upravo predstavlja bias-variance tradeoff, ukoliko povećamo bias smanjujemo varijansu i obrnuto. Smanjenjem bias-a i povećanjem varijanse izazivamo overfitting, odnosno precizno učenje i šuma, što želimo izbeći kako bi model bolje generalizovao.



Kako bi izbegli da model uči i šum, model se mora na neki način "kazniti" kada previše precizno uči šum. Kod svih modela u mašinskom učenju, kako bi model učio na trening

podacima, potrebno je izmeriti gubitke modela, odnosno koliko model greši. Gubitak se može definisati na više načina u zavisnosti od zadatka koji model rešava, ali je to uvek numerička vrednost koju želimo minimizovati i zato je problem učenja u stvari optimizacioni problem.

Zato, "kažnjavanje" modela se može ostvariti dodavanjem određene vrednosti na ukupan gubitak modela, čime model pokušava da optimizuje svoje težine za oba zadatka. Ova dodatna vrednost se najčešće računa na osnovu vrednosti težina. Kako želimo da model može i kompleksnije šablone da prepozna želimo da mu damo više parametara, odnosno više stepena slobode. Sa druge strane, ne želimo da vrednosti težina budu velike jer će to izazvati da model uči i šum.

Najpoznatije metode regularizacije su l_1 i l_2 regularizacija kao i njihova kombinacija - Elastic Net.

Ukoliko imamo model čije težine, odnosno parametri se mogu opisati matricom W i i matričnu normu definisanu kao

$$||W||, ||*|| : M^{m \times n} \rightarrow R$$

onda možemo predstaviti l_1 regularizaciju kao

$$l_1 = \lambda ||W||, \lambda \in [0,1]$$

l_2 regularizaciju kao

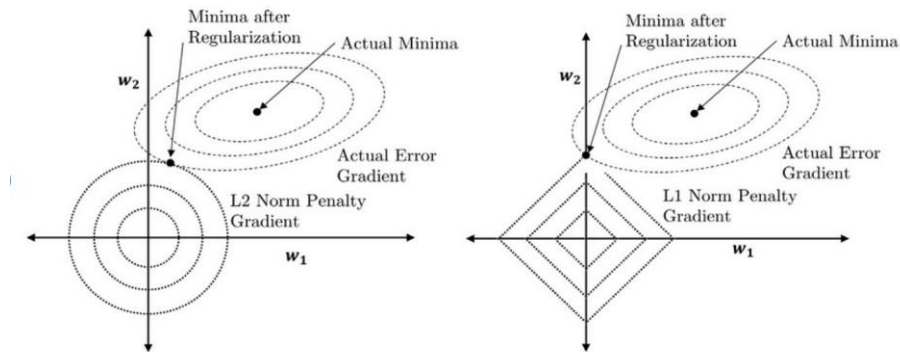
$$l_2 = \lambda ||W||^2, \lambda \in [0,1]$$

i Elastic net kao kombinaciju.

$$elasso = \lambda_1 ||W|| + \lambda_2 ||W||^2, \lambda_{1,2} \in [0,1]$$

Matrična norma služi kako bi na osnovu matrica težina dobili jedan realan broj koji predstavlja matricu. Ovde predstavljene metode regularizacije koriste matričnu normu i kvadrat norme tako što tu vrednost dodaju na ukupan gubitak. Parametri λ služe za povećanje i smanjivanje količine regularizacije. Elastic Net koristi kombinaciju ove dve metode.

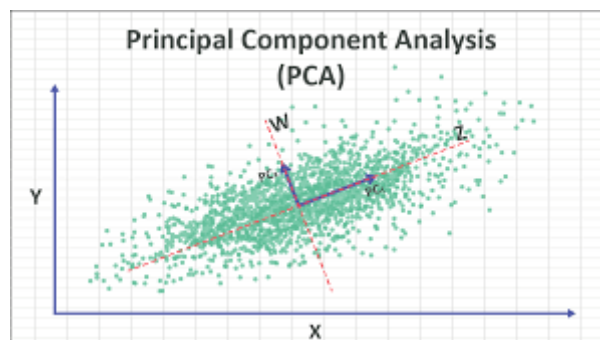
Na slici ispod možemo da vidimo izgleda loss funkcija kada se primeni regularizacija. Na slici levo se vidi uticaj l_2 regularizacije na model. Tokom treninga model pokušava da minimizuje obe funkcije, čime se izbegava overfitting tj dostizanje minimuma originalne loss funkcije. Na desnoj strani vidimo uticaj l_1 regularizacije, koja ima još jedan benefit a to je da teži da dovede vrednosti parametara što bliže nuli, a kada parametar dostigne tu vrednost može se potpuno izbaciti iz modela - time se smanjuje varijansa i ubrzava rad modela.



3.2 Otklanjanje šuma usled broja feature-a

Kao što smo objasnili u uvodu, šum može biti posledica velikog broja feature-a odnosno dimenzija. Neke dimenzije ne donose bitne podatke za zadatak modela i mogu ometati model u učenju zavisnosti među podacima. Zato je uvek dobra ideja ukloniti dimenzije duž kojih ima malo varijanse među podacima - kako bi smanjili šum ali i ubrzali trening modela. Nasumičan šum i šum usled outliera imaju mnogo veći uticaj - smanjivanjem dimenzija će se uglavnom samo malo smanjiti šum ali ćemo svakako razmotriti i ove tehnike.

Prva i među jednostavnijim metodama smanjenja dimenzija je PCA - Principal Component Analysis koja je bazirana na principima linearne algebre. Cilj PCA algoritma je da, ukoliko su podaci n dimenzionalni, odredi n vektora koji nose najviše varijanse. Zatim možemo izabrati procenat varijanse ili broj dimenzija koji želimo da zadržimo. Podaci se onda projektuju na hiperravan koja je određena pomoću $d < n$ dimenzija.



Nedostatak ove metode je u tome što je linearna. Ukoliko postoje nelinearne zavisnosti među podacima one se mogu izgubiti korišćenjem PCA. U većini slučajeva ćemo koristiti PCA s obzirom da možemo ručno specificirati koliko varijanse želimo da zadržimo - želimo da zadržimo bar 97-98%. Većim smanjivanjem postoji opasnost da izgubimo informacije.

PCA se vrši na sledeći način

1. Standardizacija

Prvi korak algoritma je standardizacija podataka, čime će vrednosti skupa podataka biti između -1 i 1, prosečna vrednost će biti 0 i standardna devijacija 1. Ovaj korak je ključan, s obzirom da je PCA osetljiv na dimenzije i raspodelu podataka.

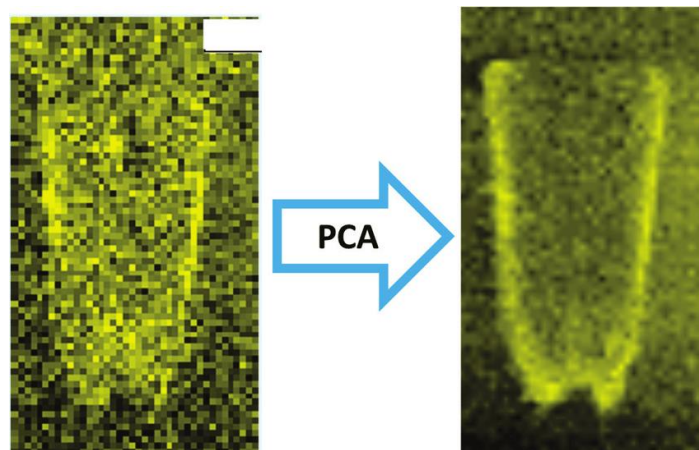
2. Računanje matrice kovarijanse

Matrica kovarijanse predstavlja matricu čiji elementi predstavljaju meru uticaja jedne promenljive skupa podataka na druge (kovarijanse).

3. Računanje sopstvenih vektora i sopstvenih vrednosti matrice kovarijanse

Računanjem sopstvenih vektora matrice kovarijanse dobićemo n vektora koji predstavljaju pravce sa najvećom varijansom. Sopstvene vrednosti će biti proporcionalni količini varijanse svakog vektora. Deljenjem svake sopstvene vrednosti zbirom svih sopstvenih vrednosti dobijamo količinu varijanse od 0-1 na osnovu koje biramo koje pravce zadržavamo.

Pomoću PCA možemo samo donekle otkloniti šum, mnogo bitnije je ukloniti nasumičan šum i šum outliera, što ćemo razmotriti u narednim poglavljima. Ispod je primer primene PCA algoritma za uklanjanje šuma na slici.



3.3 Otklanjanje outliera

Kako bi otklonili šum iz skupa podataka neophodno je ukloniti podatke koji previše odstupaju od drugih podataka u skupu - outliera. Outlieri uglavnom predstavljaju retke primerke podataka (određene specijalne slučajeve) ili usled greške tokom skupljanja podataka. Postoji više metoda za uklanjanje outlier-a a ovde ćemo razmotriti

- **Z-score**, baziran na statističkim osobinama skupa podataka,
- **Isolation Forest** - ensemble metoda za prepoznavanje outliera
- **DBSCAN** - baziran na gustini podataka, koristi se često za clustering ali i za uklanjanje outliera

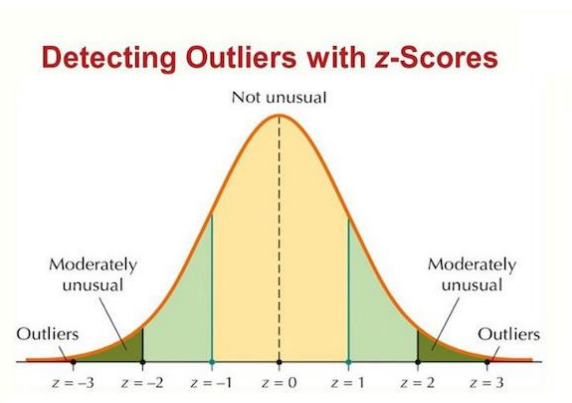
- **Autoencoder** - metoda bazirana na neuronskoj mreži, gde se odlučuje da li je podatak outlier na osnovu greške rekonstrukcije. Ovu metodu ćemo posebno obraditi s obzirom da rešava sva 3 problema kod otklanjanja šuma.

Z-score

ili standard score, je statistička metoda za otklanjanje šuma. Prvenstveno se računa standardna devijacija skupa podataka σ i prosečna vrednost μ . Zatim se za svaki podatak računa njegov z-score na sledeći način

$$z = \frac{x - \mu}{\sigma}$$

pri čemu je x podatak. Z-score označava koliko standardnih devijacija je podatak udaljen od proseka skupa. Na osnovu z-score-a možemo da odlučimo da li je podatak outlier, tako što biramo opseg prihvatljivog z-score-a (npr između -3 i 3) a ukoliko je izvan tog opsega izbacujemo dati podatak. Ukoliko podaci prate normalnu raspodelu 99.7% podataka će imati z-score između -3 i 3, a 95% između -2 i 2.



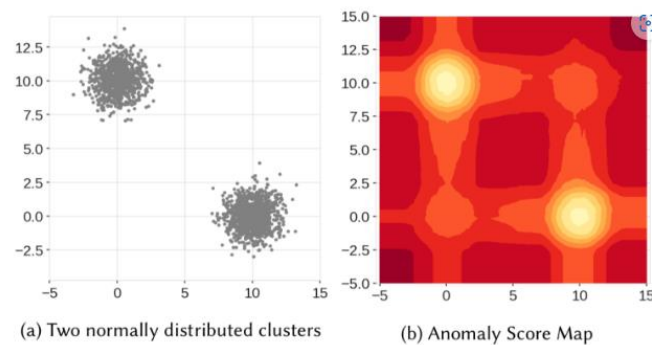
Prednost metode je u brzini izvršenja i u tome što ima dobre rezultate kod podataka sa normalnom raspodelom. Nedostatak ove metode je što pretpostavlja da podaci prate normalnu raspodelu, što nije uvek slučaj.

Isolation Forest

predstavlja metodu za otklanjanje outlier-a na osnovu skupa stabala odlučivanja. Algoritam radi na sledeći način.

1. Inicijalizuje se određeni broj binarnih stabala
2. Napravi se nasumičan podskup podataka za svako stablo
3. Stablo se grana tako što se na svakom nivou bira nasumično jedan feature na osnovu koga se vrši grananje.
4. U svakom čvoru stabla se bira nasumičan threshold za dati feature
5. Ukoliko je feature manji od thresholda, podatak ide u levo podstablo a u suprotnom u desno
6. Tako se trenira cela "šuma" nad skupom podataka

7. Nakon kreiranja se ispituje da li je neki podatak outlier tako što se propusti kroz svako stablo i računa se prosečna dužina putanje (broj koraka) kroz stablo kako bi se dobio izlaz modela
8. Na osnovu prosečnog broja koraka se dodaje labela -1 (anomalija, outlier) ili 1 (normalan podatak)

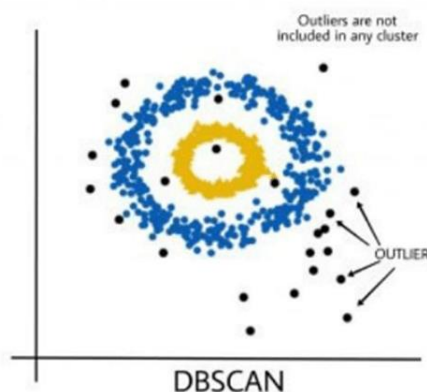


Na slici iznad vidimo rezultate algoritma, gde možemo videti prednosti i nedostatke. Prednosti algoritma su njegova efikasnost, kao i to što se pokazao korisnim za otklanjanje outliera. Međutim, nedostaci su to što moramo unapred znati meru kontaminacije skupa podataka outlierima (hiperparametar) kao i to što su usled grananja kod odlučivanja u stablu granice odlučivanja modela vrlo oštre. To dovodi do problema kao na slici - isolation forest pronalazi inliere u gornjem levom i donjem desnom uglu, ali kako zbog granica odlučivanja model smatra da i podaci u gornjem desnom i donjem levom uglu bi takođe bili inlieri

DBSCAN

Kao i Local Outlier Factor, DBSCAN je baziran na gustini podataka. Funkcioniše na sledeći način

1. Biramo hiperparametre ϵ i n . Epsilon predstavlja veličinu komšiluka u okviru koga ćemo tražiti komšije svakog podatka a n predstavlja broj komšija u ϵ okolini podatka da bi se on smatrao inlier-om.
2. Za svaki podatak gledamo koliko susednih podataka do kojih je rastojanje manje ili jednako ϵ i ukoliko je to veće ili jednako n podatak se smatra inlier-om.
3. Kada se to odredi uzimamo jedan po jedan podatak koji se smatraju inlier-ima, označimo da pripada clusteru x , i sve njegove susede u ϵ okolini takođe označimo da pripadaju clusteru x i tako ponavljamo. Kada više nemamo suseda, biramo sledeći podatak koji je inlier i ponavljamo sve dok ne clusteruje-mo sve inliere.
4. Preostali podaci se smatraju outlier-ima.



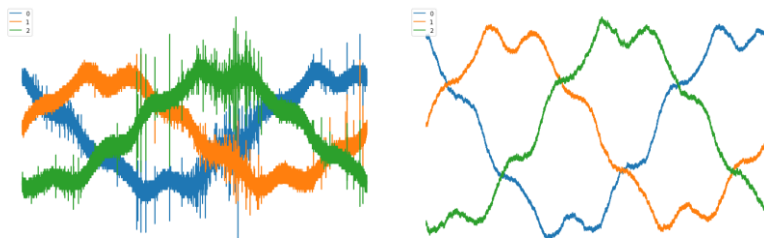
Prednosti DBSCAN-a su u tome što dobro prepoznaje outlier-e i to automatski radi u toku određivanja cluster-a. Za razliku od ostalih clustering algoritma, ne pokušava svakoj instanci da dodeli cluster već označava podatke koji odstupaju od ostalih kao outliere. Nedostatak algoritma je što je potrebno podesiti dva hiperparametra i što ne radi dobro kod skupova sa različitom gustinom u podacima, s obzirom na fiksne hiperparametre.

3.4 Redukcija nasumičnog šuma

Na kraju ćemo pogledati načine kako možemo ukloniti nasumičan šum.

Moving/Rolling Average Filter

ovaj algoritam je koristan kod time-series podataka. Baziran je na pomerajućim prozorom u okviru koga se radi prosek (ili bilo koja statistička funkcija, max, min, mean, median...). Prozor kreće od početka vremenskih podataka i pomera se udesno za po 1 korak udesno odnosno za jedinični vremenski interval, a zatim se podatak u tom trenutku u vremenu menja prosekom svih vrednosti unutar prozora. Veličina prozora se specifikira brojem n . Svaki podatak, izuzev onih vrlo blizu početka ili kraja skupa podataka, će biti članovi $n - 1$ prozora.



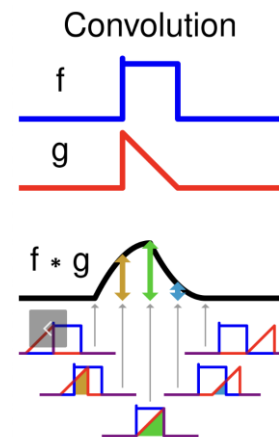
Prednosti ove metode su to što se lako računa, parametar n omogućava menjanje stepena smoothing-a podataka, kao i izbor funkcije koja se primenjuje.

Nedostatak je u tome što prvi i poslednji podaci u nizu manju utiču na smoothing, pa se te vrednosti gube donekle. Pored toga, smoothing-om se gube sitni podaci, što može da uništi male periodičnosti u podacima.

Neki problemi se mogu srediti izborom kompleksnije funkcije. Na primer, ukoliko modifikujemo algoritam tako da se koriste eksponencijalno opadajuće težine prozora čime se smanjuje uticaj podataka koji su udaljeni od centra prozora. Ovim se postiže i robusnost na outliers.

Konvolucija

Konvolucija je matematička operacija koja na osnovu dve funkcije proizvodi novu funkciju. Funkcioniše tako što se jedna funkcija "pomera" preko druge funkcije i računa se površina ispod grafova ovih dveju funkcije. Može da se smatra i kao mera preklapanja dveju funkciju tokom "klizanja" jedne funkcije preko druge. Prethodno pomenuta metoda se takođe može smatrati konvolucijom, odnosno konvolucija je generalizacija prethodno pomenute metode. Konvolucija je definisana sledećom jednačinom $(f * g)(t) = \int_{-\infty}^{\infty} f(x)g(x - \tau)d\tau$



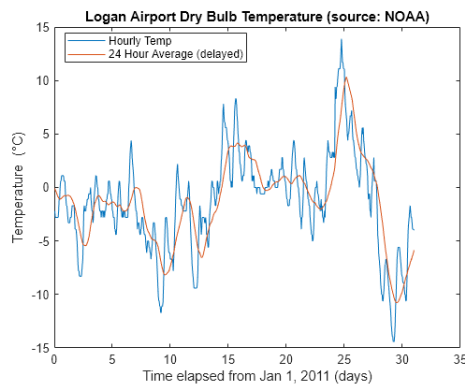
pri čemu je τ vreme. Prednosti i nedostaci ostaju isti kao sa prethodnom metodom.

Gaussian smooting

Kao i median filter, baziran je na konvoluciji odnosno sliding window principu. Gaussian smoothing predstavlja konvoluciju originalnog signala sa Gausovom funkcijom.

$$\varphi(z) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

Ova metoda je korisna kada šum može biti različitih skala. Prednosti su to što ova metoda računa novu vrednost kao težinski prosek u okviru prozora, pri čemu težine prate normalnu raspodelu, tako da će podaci bliže trenutnom podatku nositi veću težinu dok će dalji imati manju težinu. Time se dobija bolji smoothing, ali je ova metoda zbog konvolucije dosta sporija.

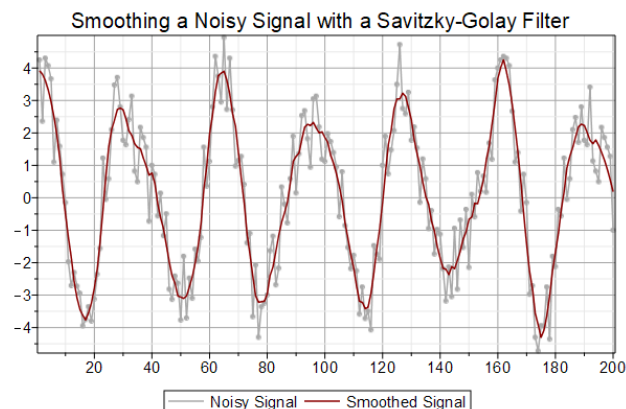


Savitzky-Golay filter

Takođe smoothing filter pri čemu se koristi polinomni filter, odnosno vrši se konvolucija originalnog signala i polinomne funkcije

$$f = a_0 + a_1x + a_2x^2 + \dots + a_n x^n$$

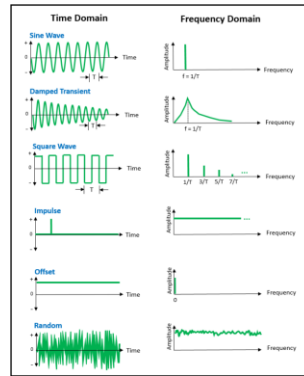
Prednost ove metode je kod podataka koji su periodični i imaju glatke trendove, odnosno ponavljanja.



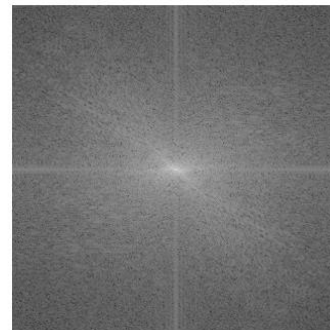
Wavelet transform - Furijeova transformacija, filteri

Wavelet transform je baziran na principima dekompozicije signala na jednostavne signale. Furijeovom transformacijom je moguće razložiti kompleksne signale kao zbir sinusnih i kosinusnih signala. Ovu transformaciju je moguće primeniti na bilo koji signal (često se primenjuje kod zvuka, slika). Na osnovu ove transformacije možemo dobiti spektar signala, tj dobiti frekvencije i amplitude signala koji sačinjavaju kompleksni signal. Odbacivanjem određenih signala koji se smatraju šumom možemo smanjiti šum. Koje signale izbaciti ili u kojoj meri smanjiti zavisi od vrste filtera koje želimo primeniti.

Na slici ispod možemo videti kako se može razložiti signal na osnovne signale korišćenjem Furijeove transformacije. Na levoj strani su kompleksni signali dok na desnoj imamo grafove koji predstavljaju koje frekvencije su prisutne u signalu i njihove amplitude.

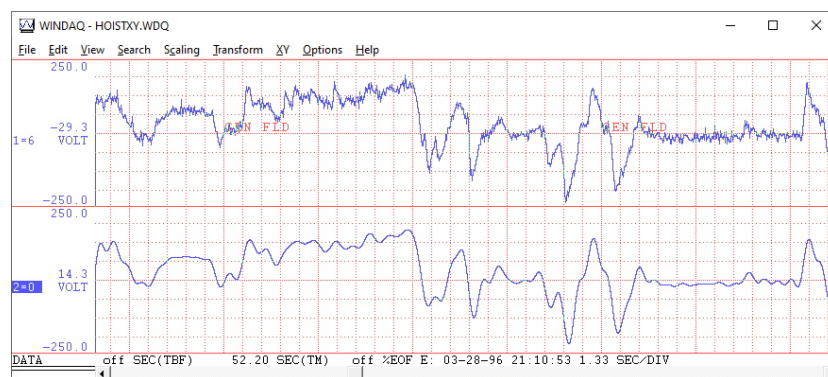


Ovakav prikaz frekvencija i amplituda možemo odrediti i za višedimenzionalne signale, npr za sliku, pri čemu imamo horizontalne i vertikalne signale. Tamniji pikseli predstavljaju signale manje amplitude, a svetliji više amplitude.



Low pass filter

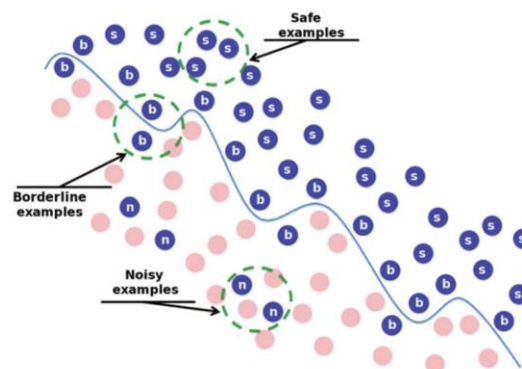
Za smanjenje šuma možemo iskoristiti Furijeovu transformaciju i low pass filter. Low pass filter zadržava signale niže frekvencije i odbacuje signale više frekvencije. Signali više frekvencije se izbacuju zato što su njima predstavljeni sitni detalji, koji najčešće predstavljaju šum. Na slici ispod možemo videti kako utiče low pass filter na signal, tj možemo videti kako uklanja sitne detalje u signalu.



3.5 Otklanjanje šuma kod alfanumeričkih podataka

Prethodno pomenute metode za redukciju šuma se mogu koristiti kod vremenskih podataka i signala. Ovde ćemo razmotriti slučaj kada imamo alfanumeričke podatke.

U slučaju da imamo alfanumeričke podatke, oni mogu biti kodirani tako da svaki podatak u skupu predstavlja n -dimenzionalni vektor $v_i \in \mathbb{R}^n$. Ukoliko je $n < 4$, ovako predstavljeni podaci se mogu grafički prikazati na grafiku ispod gde ćemo razmotriti 3 tipova podataka u skupu. Primer na slici predstavlja klasifikacioni problem, odnosno postoji hiperravan koja razdvaja dve klase podataka.



Na slici vidimo 3 tipova instanci u skupu podataka

- **Sigurne instance** - predstavljaju tipične instance podataka i one se nalaze daleko od granice odlučivanja. U ovom slučaju nema sumnje kojoj klasi pripadaju podaci.
- **Instance na granici odlučivanja** - ove instance se nalaze na samoj granici odlučivanja i njihova klasifikacija je često teška, s obzirom da male promene u vrednosti njihovih atributa mogu da ih pomere na drugu stranu granice. Količina ovakvih instanci značajno utiče na performanse modela.
- **Noisy instance** - ove instance se nalaze sa suprotne strane granice odlučivanja. Ovakve instance se mogu smatrati da pripadaju suprotnoj klasi i da su pogrešno klasifikovane od početka.

S obzirom da poslednje dve vrste instance predstavljaju šum dalje ćemo se na njih fokusirati. Zbog njihovih karakteristika ove vrste instance upravo su predstavnici dve vrste šuma kod alfanumeričkih podataka - šum labela i šum atributa.

3.5.1 Tipovi šuma kod alfanumeričkih podataka

Postoje dve vrste šuma kod alfanumeričkih podataka

1. **Šum labela** - javlja se kada je instanca u skupu podataka ima pogrešnu labelu. Postoji više uzroka ove vrste šuma, ali glavni uzrok je subjektivnost tokom unosa labelovanja podataka ili greška tokom unosa podataka. U prethodnom poglavlju

smo spomenuli noisy instance koje predstavljaju ovaj šum - podaci koji pripadaju jednoj klasi ali se ipak javljaju sa suprotne strane granice odlučivanja. Šum labela se dalje može podeliti na dve podgrupe.

- 1.1. **Kontradiktorni primeri** - podaci koji imaju iste vrednosti a različite labele
- 1.2. **Pogrešna klasifikacija** - podaci su klasifikovani drugačije od njihove prave klase
2. **Šum atributa** - javlja se kada vrednosti atributa instanci odstupaju od pravih vrednosti. Mogu nastati kao greška merenja, odnosno šuma instrumenata koji su ih merili ali mogu i predstavljati nedostajuće vrednosti.

3.5.2 Otklanjanje šuma atributa

Prvo ćemo razmotriti metode za otklanjanje šuma atributa. Ove metode mogu ili smanjiti šum promenom vrednosti atributa ili potpuno izbaciti instance iz skupa podataka za treniranje ukoliko se smatra da instanca ima previše šuma. Druga vrsta je obrađena u jednoj prethodnoj oblasti (3.3 Otklanjanje outliera) pa ćemo se zato fokusirati na smanjivanje šuma.

Prva metoda je **binning**, kojom se atributi diskretizuju, odnosno vrednosti atributa se sortiraju i zatim se dele u n bucket-a. Bucket-i mogu biti iste ili različite veličine. Na osnovu broja bucketa se određuju donja i gornja granica svakog bucket-a i podaci se zatim kodiraju. Postoji više opcija kodiranja:

- **Otklanjanje šuma pomoću mediane bucketa** - vrednost atributa se menja medianom njegovog bucket-a. Npr. bucket sa vrednostima atributa 3, 5, 6 bi bio zamenjen vrednošću 5.
- **Otklanjanje šuma pomoću proseka bucketa** - vrednost atributa se menja prosekom njegovog bucket-a. Npr. bucket sa vrednostima atributa 3, 5, 6 bi bio zamenjen vrednošću 4.667.
- **Otklanjanje šuma pomoću granica bucketa** - vrednost atributa se menja donjom ili gornjom vrednošću bucket-a, zavisno koja je vrednost bliža. Npr. bucket sa vrednostima atributa 3, 4, 5, 6 bi bio imao vrednosti 3, 3, 6, 6 nakon otklanjanja šuma.

Sorted data for *price* (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15

Bin 2: 21, 21, 24

Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9

Bin 2: 22, 22, 22

Bin 3: 29, 29, 29

Smoothing by bin boundaries:

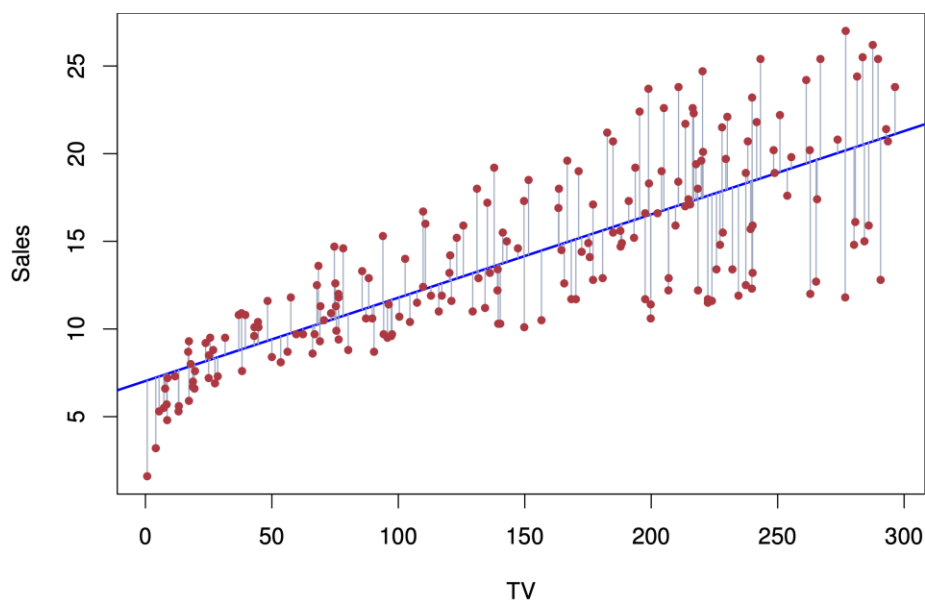
Bin 1: 4, 4, 15

Bin 2: 21, 21, 24

Bin 3: 25, 25, 34

Druga metoda je **regresija**. Regresija je metoda kojom tražimo koeficijente funkcije tako da je vrednost takve funkcije što bliža originalnim podacima nad kojim je vršeno treniranje. Na primer, u slučaju linearne regresije tražimo koeficijente prave koja će se najbliže poklopiti sa originalnim podacima. Moguće je vršiti i višestruku regresiju gde imamo više od jedne promenljive u funkciji koju tražimo. Pored linearne postoje i druge vrste regresije, kao što je polinomna regresija.

Nakon treniranja regresionog modela moguće je izvršiti zamenu vrednosti atributa korišćenjem predviđene vrednosti pomoću modela. Na slici ispod možemo da vidimo kako se može isvršiti smanjenje šuma. Nakon odabira regresione prave koja minimizuje grešku predviđanja možemo odrediti vrednost koju model predviđa i time zameniti originalnu vrednost. Ovde je bitno izabrati model čija predviđanja neće previše odstupati od originalne vrednosti (u suprotnom možemo uvesti greške) a pritom neće overfit-ovati podatke.



3.5.4 Otklanjanje šuma labela

Pored šuma atributa postoji i šum labela, odnosno šum koji nastaje usled pogrešnog labeliranja podataka. Ova greška može nastati usled ljudske greške (pogrešna klasifikacija podataka) ili usled nepreciznog merenja uređajima. Postoji više načina za otklanjanje takvog šuma, tj više vrsta filtera od kojih su najpoznatiji Ensemble filter, Cross-Validated Committees filter i Iterative-Partitioning filter.

Ensemble filter

Ensemble filter pokušava da poboljša kvalitet trening skupa podataka kao preprocesirajući korak koji se vrši pre klasifikacije. Filter vrši detekciju i eliminaciju pogrešno labeliranih instanca podataka korišćenjem skupa klasifikatora. Algoritam funkcioniše na sledeći način

- Skup podataka za trening D_T se deli na N podskupova jednakih veličina
- Uzimamo n klasifikatora pri čemu se za svaki klasifikator vrši sledeće
 - Za svaki od N podskupova, svaki algoritam se trenira na ostalih $N - 1$ podskupova, što proizvodi N klasifikatora
 - Svaki od time dobijenih N klasifikatora se koristi za klasifikaciju instanci kao tačno ili netačno labelovane. Da li je labela tačna ili ne se određuje na osnovu labele u trening skupu i predviđene labele.
- Na kraju prošlog koraka svaka instanca je označena kao tačno ili netačno labelovana od strane svakog klasifikatora.
- U skup D_N dodajemo sve instance skupa D_T koje imaju netačne labele, odnosno koje predstavljaju šum. Da li instanca predstavlja šum se određuje algoritmom za glasanje (npr. većinsko glasanje)
- Na kraju se izbacuju instance sa šumom $D_T \leftarrow D_T - D_N$

Cross-Validated Committees Filter

CVCF (Cross-Validated Committees Filter) koristi ensemble metode kako bi identifikovao i uklonio pogrešno labelovane instance iz skupova podataka kod klasifikacije. Radi na sledeći način

- Skup podataka za trening se deli na N podskupova jednakih veličina
- Za svaki od podskupova, bazni algoritam se trenira na ostalih $N - 1$ podskupova
- Zatim se ovako dobijenih N klasifikatora koristi za klasifikaciju svih instanci. Svaki klasifikator predviđa da li je instanca tačno ili netačno labelovana.
- U skup D_N dodajemo sve instance skupa koje imaju netačne labele, odnosno koje predstavljaju šum. Da li instanca predstavlja šum se određuje algoritmom za glasanje (npr. većinsko glasanje).
- Na kraju se izbacuju instance sa šumom $D_T \leftarrow D_T - D_N$

Razlika u odnosu na ensemble filter je što se za svaki podskup koristi samo jedan algoritam, dok je ensemble filter koristio n klasifikatora za svaki podskup.

Iterative-Partitioning Filter

Prethodne metode su podrazumevale da su skupovi podataka mali i da je dovoljno jednom proći kroz skup podataka kako bi se odredile pogrešno labelovane instance. Iterative-Partitioning Filter služi za određivanje i uklanjanje pogrešno labelovanih instanci kod velikih skupova podataka. Radi u koracima, odnosno iterativno ponavlja proces učenja i uklanjanja instanci dok određeni uslov nije ispunjen - ako posle određenog broja koraka s broj instanca (identifikovanih kao pogrešno labelovane) je manji od p procenta početne veličine trening skupa. Radi na sledeći način

- Skup podataka za trening se deli na N podskupova jednakih veličina. Svaki od ovih podskupova je dovoljno mali da može biti ceo procesiran induktivnim algoritmom odjednom.
- Za svaki od N podskupova se trenira jedan bazni algoritam nad tim podskupom. Na taj način dobijamo N klasifikatora.
- Svaki od N klasifikatora vrši predviđanje da li je instanca tačno ili pogrešno labelovana nad celim skupom D_T .
- U skup D_N dodajemo sve instance iz D_T za koje klasifikatori smatraju da imaju pogrešne labele. Da li instanca ima pogrešnu labelu se određuje većinskim glasanjem klasifikatora.
- U skup D_G dodajemo y procenta tačno labelovanih podataka iz skupa D_T . Cilj ovog koraka je da se brže smanji skup podataka, što je korisno kod velikih skupova podataka.
- Izbacimo dobre instance i instance sa šumom iz trening skupa $D_T \leftarrow D_T - \{D_N \cup D_G\}$
- Vrš se sledeća iteracija

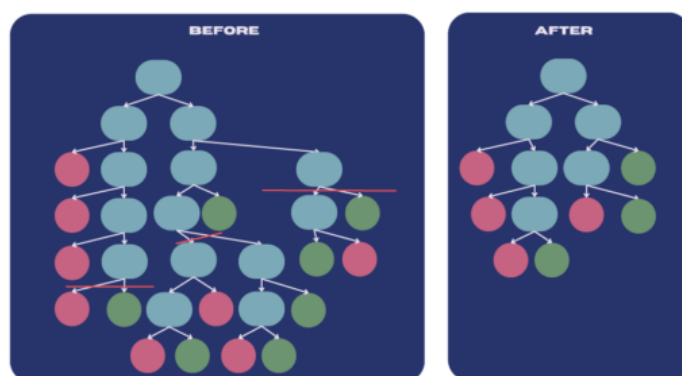
Na kraju poslednje iteracije, filtrirani podaci se dobijaju unijom preostalih instanci u D_T i dobrih instanci u D_G . Menjanjem broja iteracija filtriranja se može smanjiti ili povećati osetljivost filtera na šum.

3.5.5 Modeli otporni na šum

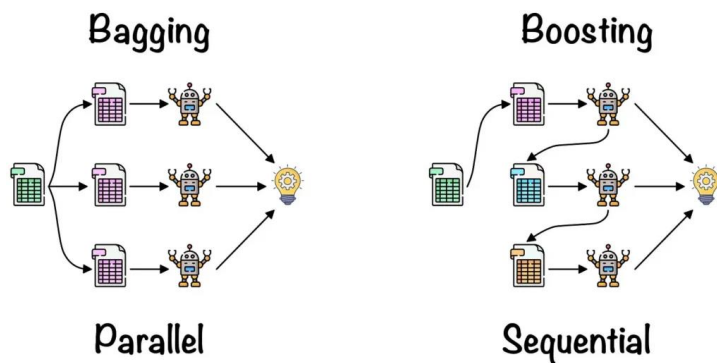
Metode koje smo obradili u prethodnom poglavlju imaju jedan veliki nedostatak a to je što uklanjaju instance iz skupa podataka koje možda nose bitne informacije. Jedno moguće rešenje jeste da se podaci ne izbacuju iz skupa već da se model učini robusnijim na šum, odnosno da algoritam bude manje osetljiv na šum u podacima. Način kojim ovo možemo obezbediti jeste ensemble learning, odnosno treniranjem više različitih modela a zatim konačno predviđanje se vrši glasanjem. U nastavku ovog poglavlja ćemo analizirati robusne klasifikatore, načine glasanja kao i strategije korišćenja više klasifikatora za predviđanje jedne labele.

Jedan primer robusnih modela su ensemble learning metode bazirane na stablima odlučivanja, kao što su random forest klasifikatori, C4.5 i noviji 5.0 i extra trees klasifikatori. Iako su stabla odlučivanja osetljiva na šum, treniranjem puno instanci modela ovog tipa dovodi do smanjenja osetljivosti tako što svaki klasifikator ima svoje prednosti, odnosno otporan je na određeni tip šuma.

Međutim, problem koji se ovde javlja jeste nestabilnost ensemble metoda, odnosno rezultati ensemble modela dosta varira u zavisnosti od parametara modela. U ovom slučaju postoje više rešenja. Jedan metod je pažljivo izabrati kriterijum podele stabla što se može postići pretragom parametara (grid search, random search...). Bolji način je pruning koji podrazumeva odsecanje dela stabla odlučivanja kojim se klasifikuje mali broj instanci, tj ovi delovi stabla su previše izolovani i najčešće predstavljaju instance sa šumom.



Ensemble metode podrazumevaju korišćenje više instanci modela istog tipa. S obzirom da želimo da instance u ensemble-u budu što razovrsnije kako bi bile osetljive na različite vrste šuma moramo menjati njihove parametre ali i instance podataka na kojima se treniraju. Postoje dva načina izbora ovih instanci podataka. Prvi je bagging, gde svaki od modela u ensemble-u se trenira nad nasumičnim podskupom podataka koji se mogu i preklapati, odnosno deliti neke instance. Druga metoda jeste boosting, gde se jedan model trenira nad svim podacima a zatim svaki sledeći trenira nad podacima koje je prethodni model pogrešno klasifikovao.



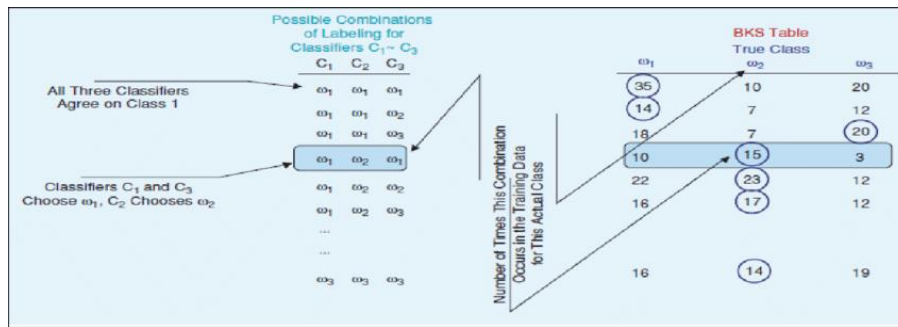
Kao još bolje rešenje se pokazalo korišćenje više različitih tipova modela u ensemble-u. Time se koriste prednosti različitih vrsta modela i ovaj metod - Multiple Classifier System (MCS) predstavlja generalizaciju ensemble metoda.

Problem koji nastaje kod MCS-a je kako izabrati koji model koristiti za inferenciju. Postoje 4 metode, pri čemu poslednja metoda se najčešće koristi i nju ćemo dalje razmotriti u nastavku.

- **Dinamički izbor klasifikatora** - kod ove metode se za svaki tip instanci određuje koji klasifikator ima najbolje performanse. Kod inferencije se bira najbolji model za ulazni podatak
- **Više stepena organizacija** - klasifikator se gradi iterativno. U svakoj iteraciji grupa klasifikatora paralelno glasa i zatim se odluka dobija kombinacijom glasova. Zatim algoritam (selektor) bira grupu klasifikatora za sledeću iteraciju na osnovu njihovih performansi u prošloj iteraciji.
- **Sekvencijalni pristup** - koristi jedan klasifikator, a ako njegova predviđanja nisu dovoljno sigurna koristi se sledeći klasifikator. Postupak se ponavlja dok se ne dobije sigurno predviđanje.
- **Paralelni pristup** - svi klasifikatori glasaju od jednom, a zatim konačni odgovor se dobija kombinacijom glasova.

Paralelni pristup se najčešće koristi i postoji više metoda kako se bira konačni odgovor na osnovu glasova.

- **Većinsko glasanje** - klasa sa najviše glasova je konačni izlaz MCS-a
- **Težinsko većinsko glasanje** - metoda slična većinskom glasanju, međutim ne cene se isto svi glasovi. Najveću težinu glasa dobijaju modeli u MCS-u koji imaju najveću preciznost.
- **Naivna Bajesova metoda** - ova metoda podrazumeva da su svi modeli u MCS-u međusobno nezavisni i da je zato konačni izlaz MCS-a ona klasa sa najvećom posteriornom verovatnoćom. Ove verovatnoće se dobijaju na osnovu konfuzione matrice klasifikatora.
- **Behavior-Knowledge Space** - funkcioniše tako što se za sve moguće kombinacije izlaza modela MCS-a određuje koji izlaz ima najveću verovatnoću da bude tačan. Na osnovu toga se formira look-up tabela. Kada se dovede ulaz svi modeli u MCS-u vrše klasifikaciju i ako njihovi izlazi se nalaze među ključevima look-up tabele dobija se odmah izlaz. Ukoliko ih nema među ključevima koristi se većinsko glasanje.



3.5.5 Postizanje otpornosti na šum MCS modela kod višeklasne klasifikacije

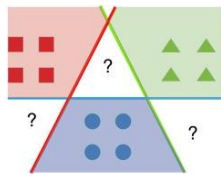
Kod problema gde postoji šum među podacima, granice između različitih klasa u skupu podataka može biti teško odrediti. Što više klasa imamo u skupu, problem klasifikacije će biti kompleksniji. Kao standardni metod za rešavanje ovog problema se javlja dekompozicija problema i to najčešće binarna dekompozicija. Binarna dekompozicija predstavlja metodu kojom se problem deli na više manjih, binarnih problema a zatim za svaki od ovih problem se treniraju modeli. Kroz više radova se pokazalo da je binarna dekompozicija jednostavna ali efektivna metoda smanjenja uticaja šuma kod višeklasne klasifikacije.

Postoje dve glavne strategije binarne dekompozicije:

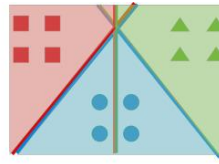
- **OvO - One versus One** - kod ove metode se trenira po jedan klasifikator za sve kombinacije parova klasa. To znači da ukoliko imamo klase $\{n_1, n_2, n_3, \dots, n_m\}$ onda treniramo klasifikatore koji će predviđati da li instanca pripada klasi n_1 ili n_2 , n_1 ili n_3 , itd. Na ovaj način dobijamo $\frac{m \cdot (m-1)}{2}$ klasifikatora. Nedostatak ove metode je kompleksnost, s obzirom da dobijamo puno klasifikatora ali po performansama je često bolji od OvA. Pored toga, trening svakog klasifikatora je brži usled manje instanci a takođe su i granice jasnije definisane.
- **OvA - One versus All** - kod ove metode se trenira po jedan klasifikator za svaku klasu, pri čemu model predviđa da li instanca pripada ili ne pripada datoj klasi. To znači da ukoliko imamo klase $\{n_1, n_2, n_3, \dots, n_m\}$ onda treniramo m klasifikatora koji će predviđati da li instanca pripada klasi ili ne pripada svakoj od klasa. Prednost metode je što imamo manje modela nego kod One-versus-One ali nedostatak je to što je trening svakog klasifikatora sporiji, s obzirom da se trenira nad celim skupom (što je kod klasifikatora čija kompleksnost raste polinomski sa brojem instanci dosta zahtevno) a takođe su granice kompleksnije.

Kako klasifikatori u ovom slučaju ne daju konačnu klasu, već da li pripada instanca jednoj ili drugoj klasi (OvA) ili da li instanca pripada ili ne pripada jednoj od klasa, potrebno je agregirati predviđanja svih modela pre donošenja odluke.

U prethodnim istraživanjima je pokazano da OvO donosi bolje rezultate usled jednostavnijih granica između klasa, balansiraniye podele trening skupa kao i kraćeg vremena treniranja modela. U praktičnom delu rada ćemo razmotriti performanse koje se dobijaju jednom i drugom metodom.



(a) Separation with OvA.



(b) Separation with OvO.

3.5.6 Uvođenje šuma u podacima

Kako bi testirali metode za otklanjanje šuma kao i robusnost modela na šum, dodavaćemo šum veštački u naš skup podataka a zatim razmatrati kako modeli reaguju na šum. Generisanje šuma može da se opiše pomoću 3 karakteristika:

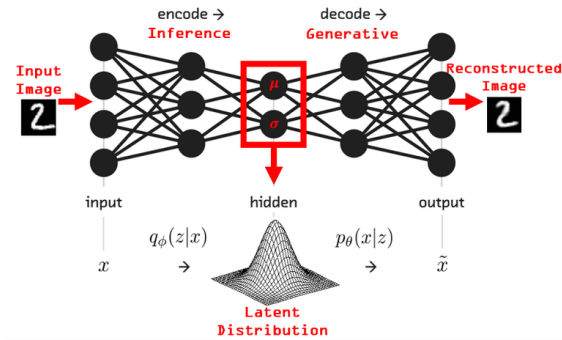
1. **Mesto gde se šum uvodi** - šum se može uvoditi među atributima skupa podataka ili među izlaznim klasama
2. **Raspodela šuma** - šum može pratiti različite podele kao što su Gausova, uniformna, Laplasova i druge.
3. **Veličina generisanog šuma** - predstavlja koliko se šuma dodaje, odnosno koliko će podaci sa šumom odstupati od početnih vrednosti.

Šum možemo dodati među labelama ili među atributima. Kod šuma među labelama korišćemo dve tehnike obrađene u [1], gde šum simuliramo nasumičnom promenom labela instanci sa određenom verovatnoćom, ili pogrešnim labeliranjem podataka većinske klase labelom druge najveće klase. Pored toga, moguće je simulirati šum među atributima dodavanjem određene količine šuma koji prati određenu raspodelu. Testiraćemo sa različitim nivoima šuma koji prate uniformnu, Gausovu kao i multionomialnu raspodelu.

3.6 Otklanjanje šuma korišćenjem autoencoder-a

Moderne tehnike otklanjanja šuma često koriste duboke neuronske mreže za otklanjanje šume, gde se posebno ističe arhitektura autoencodera. Ova arhitektura je pogotovo korisno zato što rešava više problema - smanjuje dimenzionalnost podataka, što smanjuje i šum, može da rekonstruiše nedostajuće vrednosti u skupu podataka (imputacija), vrši denoising podataka (uklanja nasumičan šum) a pored toga na osnovu rekonstrukcione greške možemo vršiti i uklanjanje outliera. Prvo ćemo pogledati i objasniti ovu arhitekturu na slici ispod.

Autoenkoder se sastoji od dva dela - enkodera i dekodera. Enkoder prima ulazni signal (podatke) a zatim sledi n skrivenih slojeva. Ulazni sloj ima onoliko čvorova koliko dimenzija ima ulazni signal a broj čvorova u skrivenim slojevima opada kako se povećava dubina mreža, sve dok se ne dođe do sloja koji ima d čvora. Ovim se u suštini vrši redukcija dimenzionalnosti, tj ulazni podaci se kompresiju od n dimenzija do d dimenzija. Zatim sledi dekode, koji radi u obrnutom smeru od enkodera. Svaki naredni skriveni sloj ima sve više čvorova, dok poslednji (izlazni) sloj ima n čvorova.

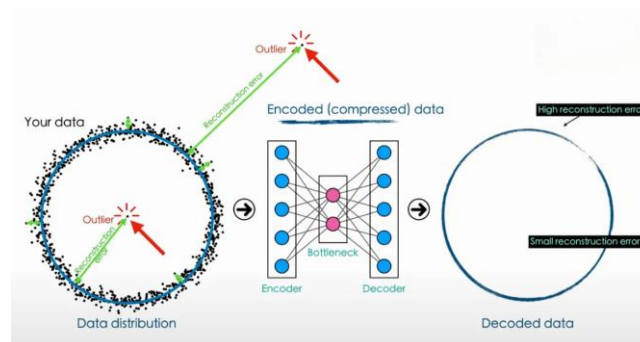


Trening autoenkodera se vrši tako što se na osnovu ulaznog skupa podatka kreira nov skup podataka, kojem je dodat šum. Taj novi skup predstavlja ulaz autoenkodera a polazni skup (bez šuma) predstavlja cilj na osnovu kojeg se meri loss mreže. Broj dimenzija d kao i broj slojeva je potrebno podesiti za što bolje performanse.

Kada je mreža istrenirana možemo je koristiti za otklanjanje šuma

- Enkoder će izvršiti redukciju dimenzionalnosti podataka i tako odstraniti deo šuma.
- Dekodiranjem ćemo dobiti podatke sa otklonjenim nasumičnim šumom, koja je mreža naučila da otkloni u toku treninga
- Ukoliko je greška rekonstrukcije velika (rekonstruisana vrednost dosta odstupa od originala) podatak se smatra outlier-om.
- Ako propustimo podatke kojima neki atributi nedostaju, rekonstrukcijom uz pomoć autoenkodera ostvarićemo imputaciju nedostajućih atributa.

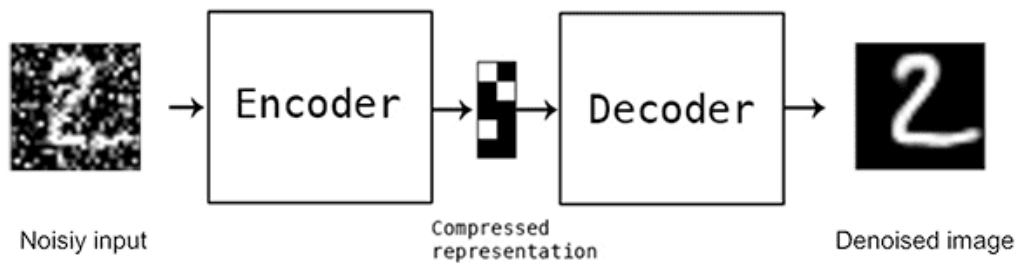
Na slici ispod možemo videti kako autoencoder-i detektuju outliere na osnovu gore opisanog algoritma.



Prednosti autoenkodera su što može dosta dobro da odstrani šum među podacima i to više vrsta šuma. Autoencoderi postižu odlične rezultate kod otklanjanja šuma. Nedostaci autoenkodera su što njihove performanse dosta zavise od izbora hiperparametara kao što su broj čvorova i broj skrivenih slojeva. Pored toga, kao i ostale

duboke neuronske mreže, treniranje autoencodera može trajati dugo pogotovo za veći broj slojeva i čvorova u mreži.

Na slici ispod možemo videti kako autoencoder uklanja nasumičan šum iz signala (slike). Naravno, nikada nije moguće potpuno ukloniti nasumičan šum ali ova arhitektura ima odlične rezultate u ovom zadatku.



5. Literatura

- [1] <https://www.statisticshowto.com/statistical-noise/>
- [2] <https://www.geeksforgeeks.org/how-to-handle-noise-in-machine-learning/>
- [3] <https://dataheroes.ai/glossary/noise-in-machine-learning/>
- [4] <https://www.iguazio.com/glossary/noise-in-ml/>
- [5] <https://www.deepchecks.com/glossary/noise-in-machine-learning/>
- [6] [https://en.wikipedia.org/wiki/Noise_\(signal_processing\)](https://en.wikipedia.org/wiki/Noise_(signal_processing))
- [7] <https://builtin.com/data-science/l2-regularization>
- [8] <https://ieeexplore.ieee.org/document/9195150>
- [9] https://workshop.dipy.org/documentation/1.7.0/examples_built/04_preprocessing/denoise_mppca/#:~:text=The%20basic%20idea%20behind%20the,that%20are%20classified%20as%20noise.
- [10] https://www.researchgate.net/figure/Example-of-noise-reduction-by-data-processing-using-principle-component-analysis-on-a-Mn_fig3_335402543
- [11] <https://www.sciencedirect.com/topics/computer-science/local-outlier-factor>
- [12] <https://www.analyticsvidhya.com/blog/2021/07/anomaly-detection-using-isolation-forest-a-complete-guide/>
- [13] <https://www.engati.com/glossary/dbscan>
- [14] <https://medium.com/@pp1222001/what-are-outliers-outliers-in-machine-learning-3a4804ab690>
- [15] <https://www.kaggle.com/code/residentmario/denoising-algorithms>
- [16] https://en.wikipedia.org/wiki/Fourier_transform
- [17] <https://www.dataq.com/blog/unrelated-interesting/using-a-low-pass-filter-to-clean-up-noisy-signals/>
- [18] <https://dagster.io/glossary/data-denoising>
- [19] <https://dataheadhunters.com/academy/handling-noisy-data-smoothing-and-filtering-techniques/>
- [20] <https://www.analyticsvidhya.com/blog/2023/07/unveiling-denoising-autoencoders/>